

SM2



Overview: What Are We Building?

A **simple terminal program** that:

- Shows flashcards (question and answer)
- Asks how well you remembered the answer (rate from 0 to 5)
- Uses the **SM2 algorithm** to decide when to show the card again

This makes your studying more efficient — cards you remember well will appear less often, and harder ones will come back sooner.



Step-by-Step Plan

◆ Step 1: Create a Dart console project

Goal: Set up a basic console app that runs from your terminal.

What to do:

1. Open your terminal.
2. Run this command:

```
dart create -t console sm2_flashcards
```

3. Go into the project folder:

```
cd sm2_flashcards
```

4. Open this folder in your code editor.
-

◆ Step 2: Understand what data a flashcard needs

Goal: Know what information each flashcard should store.

Each flashcard will have:

- `question` – the thing you’re trying to learn
- `answer` – the correct answer

- `repetitions` – how many times you remembered it correctly
- `interval` – how many days until the next review
- `easeFactor` – how easy this card is for you (starts at 2.5)
- `nextReviewDate` – when you should review it again

This info is used to decide **when** you'll see the card next.

◆ Step 3: Think about how the SM2 algorithm works

Goal: Understand the logic that updates the flashcard.

When you review a card, you give it a score from **0 to 5**.

- If you score it **0–2** → you forgot or struggled
 - Reset `repetitions` to 0
 - Set `interval` to 1 (review again tomorrow)
- If you score it **3–5** → you remembered it
 - Increase `repetitions` by 1
 - If it's the **first correct review** → `interval` = 1
 - If it's the **second** → `interval` = 6
 - If it's the **third or more** → `interval` = `previous interval` × `easeFactor`
- Update the **easeFactor**:

```
EF = EF - 0.8 + (0.28 × quality) - (0.02 × quality × quality)
```

- Minimum EF allowed = 1.3
- Calculate the **next review date**:

```
nextReviewDate = today + interval days
```

✅ This algorithm helps space out cards based on how well you remember them.

◆ Step 4: Make a class to represent a flashcard

Goal: Create a reusable object that stores and updates flashcard data.

This class should:

- Store all the values listed in Step 2
- Have a method like `updateReview(int quality)` that applies the SM2 logic from Step 3

This keeps your code clean and organized — each card knows how to update itself.

◆ Step 5: Create a small list of flashcards manually

Goal: Have a few cards ready for testing.

Just create 2–3 flashcards in your code manually with simple questions.

Later, you can expand this to loading cards from a file, but don't worry about that yet.

◆ Step 6: Make the main loop of your program

Goal: Show each card, get user input, and update the card.

For each flashcard:

1. Check if the card is **due today** or earlier:
 - `if (card.nextReviewDate <= today)`
2. If yes:
 - Show the question
 - Wait for user to press Enter
 - Show the answer
 - Ask the user to rate their recall (0–5)
 - Pass that score into the card's `updateReview()` method

This is the **main user experience** of your flashcard app.

◆ Step 7: Work with user input in the terminal

Goal: Interact with the user in the terminal.

You need to:

- Use `print()` to show messages
- Use `stdin.readLineSync()` to read user input
- Convert input string to a number using `int.tryParse(...)`
- Validate that the number is between 0 and 5

If the user types something invalid, ask again.

◆ Step 8: Show updated flashcard info after review

Goal: Let the user see how the card changed.

After reviewing each card, print:

- How many times it's been recalled correctly
- What the new interval is (in days)
- When the next review is

This gives the user feedback and also helps you test the algorithm.

◆ Step 9: (Optional) Add saving and loading flashcards

Goal: Save your flashcards to a file so progress isn't lost.

You can:

- Use Dart's `dart:io` to read/write files
- Use `dart:convert` to turn your flashcard object into JSON
- Save flashcards to a file at the end of the session
- Load flashcards from the file at the start of the program

But only do this after your main logic works well.

Recap – What You're Building

You are creating a small, helpful study tool that:

- Uses flashcards
 - Lets you rate how well you remember
 - Updates each card with a smart algorithm (SM2)
 - Tells you when to review the card again
-

Next Steps (Checklist to Follow While Building)

You can follow this checklist while coding:

1.  Create a Dart console project
2.  Define what data a flashcard needs
3.  Understand the SM2 logic
4.  Make a `Flashcard` class with `updateReview()` method

5.  Create 2–3 flashcards manually
6.  Loop through flashcards and check if they're due
7.  Show the question and answer
8.  Ask the user to rate their memory (0–5)
9.  Call `updateReview()` with that rating
10.  Show updated info about the card
11.  (Optional) Save/load flashcards using JSON